
Computer lab exercises 3

TANA21/22: BERÄKNINGSMATEMATIK

Andrew R. Winters

HT1 2026

Name: _____ Group: _____

LiU ID: _____

E-mail: _____

Name: _____ Group: _____

LiU ID: _____

E-mail: _____

Approved: _____ Signature: _____

Corrections: _____

The goal of this set of computer exercises is to explore:

1. Discrete differentiation using finite differences and their accuracy.
2. Polynomial interpolation.
3. Constructing interpolating polynomials for different functions and data sets.
4. Comparing the quality of global versus piecewise interpolation.

Additionally, the tasks in this set of computer exercises are helpful for aspects of the second and third mini-projects.

3.1 Approximating the derivative of a function

Consider a discretization of the unit interval $[0, 1]$ into $N + 1$ equidistant points $x_i = i/N$, $i = 0, 1, \dots, N$. Recall that the derivative $f'(x)$ of a differentiable function $f(x)$ can be approximated using the central difference stencil

$$f'(x_i) \approx \frac{f(x_{i+1}) - f(x_{i-1}))}{2h}.$$

Note, however, that this formula is not applicable at the boundary nodes x_0 and x_N . To approximate the derivative in the boundary nodes we need to modify the difference stencil. A natural choice is to use the one-sided approximations

$$\begin{aligned} f'(x_0) &\approx \frac{f(x_1) - f(x_0)}{h} \\ f'(x_N) &\approx \frac{f(x_N) - f(x_{N-1}))}{h} \end{aligned}$$

- (a) Let $N = 5$. Construct a matrix $\mathbf{D}_x$ based on the stencils above, such that if

$$\mathbf{f} = [f(x_0) \quad f(x_1) \quad \cdots \quad f(x_N)]^T,$$

then $(\mathbf{D}_x \mathbf{f})_i \approx f'(x_i)$ for $i = 0, 1, \dots, N$.

Tips: Use the `np.diag` command. See Exercise 2.2 from Lab 2 for an example.

- (b) Compute $\mathbf{D}_x \mathbf{f}$ for the functions $f(x) = 1$, $f(x) = x$, and $f(x) = x^2$ and compare with the exact derivatives. For which of these functions is your derivative matrix approximation exact?

- (c) Modify your treatment at the boundary nodes using a biased stencil:

$$f'(x_0) \approx \frac{-3f(x_0) + 4f(x_1) - f(x_2))}{2h} \qquad f'(x_N) \approx \frac{f(x_{N-2}) - 4f(x_{N-1}) + 3f(x_N))}{2h}$$

Repeat task (b). How does the exactness of this new differentiation matrix change?

- (d) Consider the function $f(x) = \exp(\sin(4x))$. Use the chain rule (kedjeregel) to compute the analytical expression for $f'(x)$. Using the boundary stencil from part (a), compute the maximum of the point wise error

$$\max_{0 \leq i \leq N} |f'(x_i) - (\mathbf{D}_x \mathbf{f})_i|$$

for $N = 10$, $N = 100$, and $N = 1000$. Repeat this using the boundary stencil from part (c). Report the errors and their ratio, e.g., $\text{error}_{N=10}/\text{error}_{N=100}$.

N	error for $\mathbf{D}_x$ from (a)	Error ratio	error for $\mathbf{D}_x$ from (c)	Error ratio
10		—		—
100				
1000				

How does the difference stencil at the boundary affect the errors and the approximation order?

3.2 Polynomial interpolation with the Lagrange basis

The Lagrange basis polynomials have the ability to create a global interpolating polynomial for a given set of function data. First, with pen-and-paper, investigate the accuracy of using an interpolating polynomial to approximate a polynomial function.

Recall that the Lagrange polynomial basis functions are

$$\ell_i(x) = \prod_{\substack{j=0 \\ j \neq i}}^n \frac{x - x_j}{x_i - x_j}, \quad \text{where } i = 0, \dots, n$$

for a given set of interpolation nodes $\{x_i\}_{i=0}^n$. The interpolating polynomial of degree n in Lagrange form is

$$p_n(x) = \sum_{i=0}^n y_i \ell_i(x)$$

where y_i , $i = 0, \dots, n$ is the set of function data that is either provided or computed by evaluating a function at the interpolation nodes.

- (a) Below are three data points for the function $f(x) = 2x^2$. In the space provided, construct the three Lagrange basis polynomials and determine the second degree interpolating polynomial for the data.

x	-1.0	0.0	1.0
y	2.0	0.0	2.0

$$\ell_0(x) =$$

$$\ell_1(x) =$$

$$\ell_2(x) =$$

$$p_2(x) = \sum_{i=0}^2 y_i \ell_i(x) =$$

How accurate is the polynomial interpolation for the function $f(x) = 2x^2$?

- (b) For this task, you will create a python function to evaluate the interpolating polynomial $p_n(x)$ written in the Lagrange basis at a particular value $x = z$. The goal is to translate the mathematical formulas we have from the page into a form that the computer understands.

Consult the formulas on the previous page for the Lagrange basis functions $\ell_i(x)$ and the interpolating polynomial $p_n(x)$. Organize some of your thoughts on this task before the implementation by answering the following questions:

1. What information must be given as input to the function for the computation?
2. What information is handed back as output from the function to the user?
3. What is the data dependency? That is, are there pieces of the mathematical formula that must be computed before others?

From your answers above, fill in the missing information to write pseudocode for your implementation.

```
// Pseudocode to compute  $p_n(x)$  written in the Lagrange basis at the point  $x = z$ 

// INPUT:  $z$ , _____

// OUTPUT:  $p_n(x)$ 

 $p_n(x) =$  ____

for ____ = ____ to ____ do

    for ____ = ____ to ____ do

        _____

    end // loop for the product

    _____

end // loop for the sum
```

Finally, download the skeleton `Lagrange_interpolation.py` file from the course documents. Use your pseudocode as a guide and implement the evaluation of the interpolating polynomial $p_n(x)$ in Lagrange form evaluated at the value $x = z$.

(c) Validate your implementation of the Lagrange interpolating polynomial by examining the two functions:

1. $f_1(x) = 2x^2$ on three uniform interpolation nodes ($n = 2$) on the interval $[-1, 1]$.
2. $f_2(x) = x^3 + 2x - 3$ on four interpolation nodes ($n = 3$) on the interval $[-1, 1.5]$ using the data

x	-1.0	0.0	1.0	1.5
y	-6.0	-3.0	0.0	3.375

To visualise the interpolating polynomial you will need to apply your `Lagrange_interpolation.py` many times to an enriched set of values in the given interval. This is because the routine written in (b) is designed to evaluate the interpolating polynomial at a *single point*.

OBS! You will use this function several times in this lab.

To aid in this sampling of the interpolating polynomial create a short function `build_Interpolation.py` to evaluate your Lagrange evaluation routine for M points (typically large like $M = 150$) on the interval of interest and return the evaluations in the vector `L`. For instance, if the data set $\{(x_i, y_i)\}_{i=0}^n$ is stored in `xVals` and `yVals`:

```
import numpy as np
from Lagrange_interpolation import Lagrange_interpolation

def build_Interpolation(xVals, yVals, M):
    X = np.linspace(xVals[0], xVals[-1], M)
    L = np.zeros((M,))
    for k in range(M):
        L[k] = Lagrange_interpolation(X[k], xVals, yVals)

    return X, L
```

Use `matplotlib` to plot the original function, the given interpolation nodes, and your interpolating polynomial `L` evaluated on the enriched set of points `X`. For convenience, save the nodes and evaluated polynomials for the two functions in separate arrays, e.g., `X1`, `L1` and `X2`, `L2`. Do this visual comparison for the data and function $f_1(x)$ and $f_2(x)$.

How do your interpolating polynomials compare *visually* to the exact functions $f_1(x)$ and $f_2(x)$?

Calculate the pointwise error between the interpolating polynomial evaluated on the enriched set of points and the exact function. Is this error what you expect given the exactness of the interpolating polynomial and your calculations in (a)?

OBS! This assumes that the exact function is defined as a function, e.g., `func1 = lambda x: 2*x^2`.
`np.linalg.norm(func1(X1) - L1, ord=np.inf) =`

`np.linalg.norm(func2(X2) - L2, ord=np.inf) =`

3.3 Polynomial interpolation for complicated function

This exercise illustrates how interpolation can model complex mathematical functions with simple polynomials. To do so, we examine the *gamma function*, which is a generalization of the factorial for all positive real numbers. It is defined by

$$\Gamma(x) = \int_0^{\infty} t^{x-1} e^{-t} dt, \quad x > 0.$$

For integer arguments n , the gamma function has values of the factorial $\Gamma(n) = (n-1)!$. Thus, constructing a polynomial to interpolate the data points

x	1	2	3	4	5
y	1	1	2	6	24

will approximate the gamma function on the interval $[1, 5]$.

- (a) Use your interpolation routine `Lagrange_interpolation.py` from Exercise 3.2(b) to create a polynomial of degree four that interpolates these data points and plot the result on the enriched set of data points. To do so, you can adjust the plotting commands from the previous Exercise.

How does your interpolating polynomial compare to the exact gamma function? **OBS!** SciPy has a function `gamma` that you can use for this check via `from scipy.special import gamma`.

- (b) Compute the error between your `Lagrange_interpolation.py` routine evaluated at the point $x = 1.5$, which lies *within* the interval $[1, 5]$, and the exact value of the gamma function $\Gamma(1.5) = \sqrt{\pi}/2$. Is the interpolating polynomial a good approximation to $\Gamma(1.5)$?

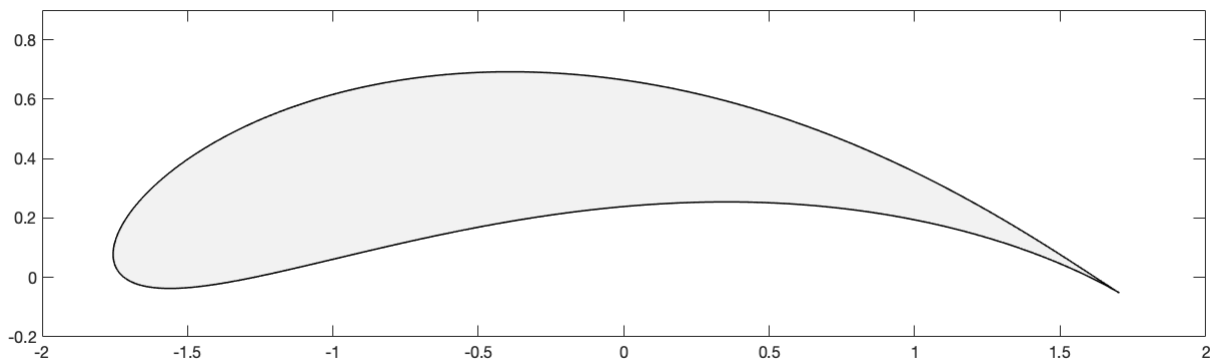
- (c) Compute the error between your routine evaluated at the point $x = 6$, which lies *outside* the interval $[1, 5]$, and $\Gamma(6) = 5! = 120$. Is the interpolating polynomial a good approximation to $\Gamma(6)$?

- (d) What conclusion can you draw from (b) and (c) about the interpolating polynomial and its ability to approximate a function? Is there a certain data range where the interpolation is more reliable?

3.4 Piecewise interpolation with splines versus global polynomial interpolation

This task explores the usefulness of interpolating splines versus a global interpolating polynomial. We add some context for this task to motivate the problem.

Suppose you are employed at SAAB and work as a numerical analyst in their aircraft design division. The design engineers have come up with a new Joukowski-type airfoil for testing and analysis.



You are instructed to create a polynomial representation of the top of this airfoil from a small set of data points provided to you by the design department. They want the numerical representation to be accurate.

- (a) Download and run the script `joukowsky_airfoil_image.py` available in the course documents. This reproduces the figure above.
- (b) Download the file `joukowsky_airfoil_data.py` and import the data. Plot *only* the data points using the command

```
ax.plot(x_joukow, y_joukow, marker='o', linestyle='None', color='b',
        markerfacecolor='b', markersize=8)
```

This will print the data locations as blue dots along the top of the airfoil. How are the data points distributed? Why, for this application, are more data points needed in certain regions?

- (c) Use your `Lagrange_interpolation.py` routine to create a global interpolating polynomial for the given data set and plot it on an enriched set of points in the interval $[-1.775, 1.695]$

```
X, Air = build_Interpolation(x_joukow, y_joukow, 150)
ax.plot(X, Air, '--r', linewidth=2)
```

- How does this global representation look compared to the airfoil shape?
- Did the interpolation work “properly”?
- How does the polynomial behave at the data points? What about *between* the data points?

- (d) Create an interpolating cubic spline to capture the top profile of this airfoil design. Use the `spline` function from `scipy` to do this for the given data set `x_joukow`, `y_joukow` on the enriched set of points `X` (from above) with the commands:

```
from scipy.interpolate import splrep, splev
b_spline = splrep(x_joukow, y_joukow)
s = splev(X, b_spline)
```

The function `splev` returns a vector `s` that you can plot against the enriched set of values in `X`. Plot this interpolating spline data against the reference image with the command:

```
ax.plot(X, s, '-.g', linewidth=2)
```

- How does this spline representation look compared to the airfoil shape?
- How does the spline behave at the data points? What about *between* the data points?

- (e) For this application, is there an advantage you have observed using piecewise versus global interpolating polynomials? Why?